Nicholas Cabrera
Ethan Patterson

a) The typical counting inversions algorithm will take advantage of merge sort's divide and conquer approach by incrementing the number of inversions by the number of elements remaining in the "left" array whenever an element from the "right" array is added. This must be adapted for counting weights by preprocessing "suffix" weights for the left array. This preprocessing runs in O(n) time by filling in an array of equal length to the left array from right to left, summing the elements. For example, if the "left" array is [3, 7, 8], then the suffix array would be [3+7+8, 7+8, 8]. This allows us to get the sum of all the remaining elements in an array in O(1) time while merging from the right array. In terms of workflow, once the left and right sides are both sorted, they will be merged. When merging from the right, the algorithm checks what the index is for the left array, and pulls the correct sum from the same index in the suffix array. Then it will use the formula: weight += suffix_array[i] + (num_remaining_on_left_side * number_from_right) to calculate the correct weight to add. It will continue to do this until the array is fully sorted and merged.

b)

```
mergeAndCount:
        Sum = 0
        Precompedsizes = []
        For (k = leftarray.size-1; k>=0; k–)
                Sum += leftarray[k]
                precompedsizes.push(sum)
        merge_sort(array)
        …else{ // case where we pull from the right array during the merge
        newArray[arrayPos] = rightArray[rightPos]
        rightPos++
        if(leftArrayPos < leftArray.size)
                inversionCount += precompedsizes[leftpos] + (leftArray.size - leftpos) *
newArray[arrayPos]
}

sortAndCount:
        Left = sortAndCount(lefthalf(array))
        Right = sortAndCount(righthalf(array))
        Merged = mergeAndCount(left, right)
        merged.inversioncount = left.inversioncount + right.inversioncount +
merged.inversioncount

        Return merged
```

c) Mergesort preserves the ordering when recursively sorting. Not completely, but enough to know “this half is before that half”, and when we break it down to such a fine degree that each number is its own half, we’ll test every possibility. Thus, when adding two arrays during the merge(left and right halves, with the left being the side closer to 0), the left half completely comes before the right half in the original ordering, thus, if any values are used from the right half, all the values that will come after it(sorting wise) from the left half are guaranteed to be inversions.

We can then further simplify it by pre calculating the cumulative sums at every point in the left array, as the cumulative sum of the current remaining left values times the current right value in question times the number of values remaining in the left, will distribute out to be the sum of the inversions.

d) A running time for this algorithm is O((n/2) + nlogn), or more accurately O(nlogn).

e) The running time for this algorithm is O(nlogn) as we follow the same divide and conquer approach as merge sort, and the suffixes are all preprocessed once when we have the left arrays. As n/2 is a lower order than nlogn, it is dropped in favor of the higher order term which comes from the known merge sort runtime.